MSML610: Advanced Machine Learning

Lesson 09.5: Kalman Filter

Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- AIMA 14: Probabilistic reasoning over time
- https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

- ***Dynamic Bayesian networks***

Dynamic Bayesian Networks (DBNs)

- DBNs extend Bayesian networks to model temporal processes
- Main idea
  - "Unroll" the model over time
  - Capture intra-slice (within time) and inter-slice (across time) dependencies
- Each time slice includes:
  - State variables X_t
  - Evidence variables E_t
- Assumptions
  - First-order Markov process: current state depends only on the previous state
  - First-order sensor Markov process: evidence depends only on current state
  - Stationarity: each time slice is the same, both structure and parameters do not change over time
    - Structure and CPTs (Conditional Probability Tables) are the same across slices (time-homogeneous model)

DBNs vs HMMs

- DBNs generalize Hidden Markov Models (HMMs)
- HMMs are a special case with a single hidden and evidence variable per time step
- DBNs model more complex systems than HMMs by:
  - Using multiple state variables
    - Enables modeling large systems like robot localization with many state components
  - Exploiting sparse connections among variables yielding compact model
    - HMM: transition matrix of size $O(d^{2n})$
    - DBN: size $O(nd^k)$ with k bounded parents per variable

DBNs vs Kalman Filters

- DBNs generalize Kalman filters
- Every Kalman filter can be represented in a DBN with:
  - Continuous variables
  - Linear / Gaussian conditional distributions
- Not every DBN can be represented by a Kalman filter, since:
  - DBN variables can mix discrete/continuous and non-Gaussian
  - Allow arbitrary conditional dependencies among variables
- **Pros of DBNs**
  - DBNs are applicable to broader domains including:
    - Fault diagnosis in networks
    - Complex system monitoring
- **Pros of Kalman filters:**
  - Optimal for linear systems with Gaussian noise
  - Support exact inference, DBNs often require approximate methods

Constructing a DBN

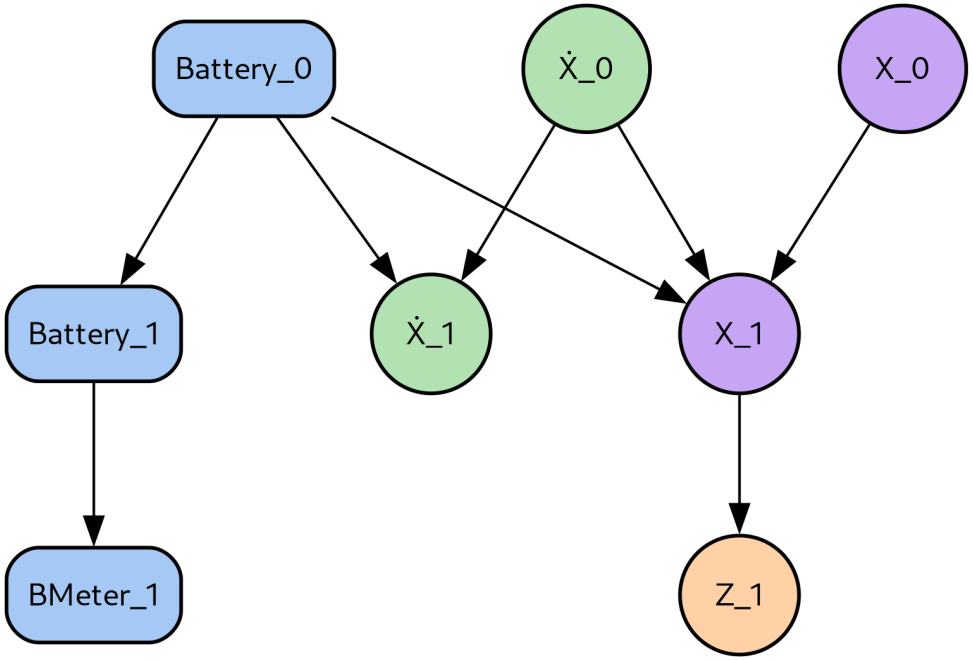
- Key components of a DBN
  - Prior distribution of state $\Pr(X_0)$
  - Transition model $\Pr(X_{t+1}|X_t)$
  - Sensor model $\Pr(E_t|X_t)$
  - Transition and sensor models are time-homogeneous
- Network topology includes:
  - Intra-slice topology
  - Inter-slice links

DBN Example: Tracking a Robot (1/3)

- **Problem:**
  - Tracking a robot moving randomly on a line X over time
- **Initial model:**
  - Position X_t and velocity $\dot{X}_t$ as state variables
  - Update via Newton's laws
  - Easy to generalize for 2d or 3d by using a $\underline{\boldsymbol{X}}_t$
- **Issue:**
  - Velocity changes over time
  - Battery exhaustion affects velocity systematically
  - Effect depends on cumulative energy use
  - Violates the Markov property (future depends on full history)
- **Solution:**
  - Include battery level $Battery_t$ in the state X_t
  - Restores the Markov assumption
  - Allows motion prediction considering energy constraints
  - Enables coherent reasoning about motion and power consumption over time
- **New requirement for state:**
  - $S_t = (X_t, \dot{X}_t, \text{BatteryLevel}_t)$
  - $E_t = (\text{GPS}_t, \text{BMeter}_t)$

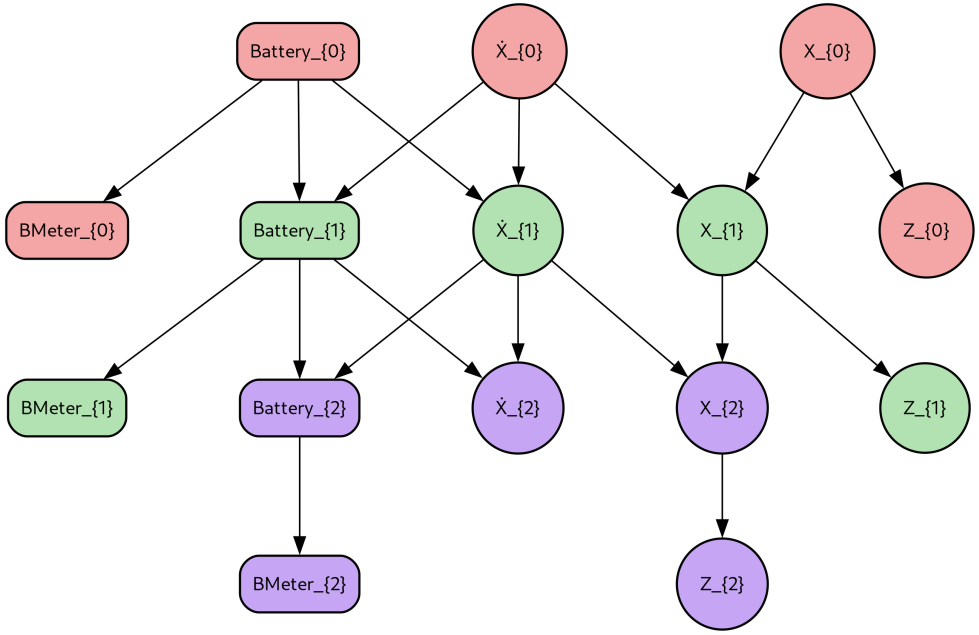
DBN Example: Tracking a Robot (2/3)

- The DBN structure models both intra-slice (within time) and inter-slice (across time) dependencies
- Intra-slice dependencies:
  - Position X_t influences velocity $\dot{X}_t$
  - BatteryLevel_t influences velocity $\dot{X}_t$
  - Battery_{t+1} depends on Battery_t and $\dot{X}_t$
  - BMeter_t depends on Battery_t
  - GPS_t depends on X_t
- Inter-slice dependencies:
  - Position X_{t+1} depends on Position X_t and velocity $\dot{X}_t$
  - Velocity $\dot{X}_{t+1}$



DBN Example: Tracking a Robot (3/3)

- **Replicate for Multiple Time Slices**:
  - Create slices for $t = 0, 1, 2, \ldots$ with the above variables and dependencies
  - Group each time slice vertically or horizontally for clarity
- **Unrolling**:
  - Visualize the full DBN by unrolling these slices over the desired number of time steps (e.g., three slices for $t = 0, 1, 2$)



Inference in DBNs

- DBNs are Bayesian networks and we can use the same inference algorithms
  - "Unroll" the DBN over time (i.e., replicate slices for each time step) and apply standard BN inference
  - We can't unroll "forever", but we limit to a certain number of slices to approximate a fixed amount of time dependency
- Use recursive methods to get a constant time and space update complexity
  - Variable elimination with temporal ordering
  - At time step $t+1$ add slice $t+2$ and remove slice t so one has always two slices to do inference
  - Maintains constant memory by keeping only two slices at a time
- Complexity:
  - Exponential in number of state variables ($O(nd^{n+k})$)
  - More efficient than full HMM representation ($O(d^{2n})$)
- Even though we can use DBNs to represent very complex temporal processes with many sparsely connected variables, we cannot reason efficiently and exactly about those processes
  - The prior joint distribution over all the variables is factorizable into its constituents CPTs

Approximate Inference in DBNs

- Particle Filtering:
  - Represent belief state with weighted samples (particles)
  - Steps: propagate, weight, resample
- Benefits:
  - Focuses computation on high-probability regions
  - Maintains manageable memory and time per step
- Challenges:
  - Approximation error
  - Sensitive to transition and observation model assumptions
- Used when exact inference is computationally impractical
- Real-world application: robot localization, speech recognition

DBN to Represent Changing Model

- We can model the fact that the system can change over time
  - Transient failure: a sensor reads wrong measures
  - Persistent failure model: we can model it with additional variables (e.g., *SensorBroken*)

DBN: Inference

- We can unroll the DBN and get a BayesNet and then perform exact or approximate inference with the known methods (e.g., MCMC)

DBN: Optimization for Inference

- Many optimizations are possible, e.g.,
  - Instead of running each sample through the entire DBN one can run all the samples evaluating one slice at a time to compute the posterior distribution